INTRODUCTION TO COMPUTER PROGRAMMING

LEARNING OBJECTIVES

At the end of this lesson, you should be able to:

- identify the phases of the software development process.
- discuss the importance of planning, analysis, design, implementation, testing, and maintenance.
- differentiate between various software development methodologies.
- differentiate between functional, declarative, and logical programming paradigms.



INTRODUCTION

- Computer programming is the art and science of creating software applications by writing sets of instructions that computers can understand and execute. It involves the process of designing, coding, testing, and maintaining programs that enable computers to perform specific tasks or solve problems.





INTRODUCTION

- Programming languages, such as Java, Python, C++, and JavaScript, serve as the means of communication between humans and computers.
- Programmers use these languages to translate their ideas into functional software that can range from simple scripts to complex applications and systems.





INTRODUCTION

- Programming encompasses a wide range of domains, including web development, mobile app development, game design, artificial intelligence, data analysis, and more.
- It requires logical thinking, problem-solving skills, creativity, and attention to detail.
- Learning to program opens doors to innovation and empowers individuals to create solutions that shape our digital world.



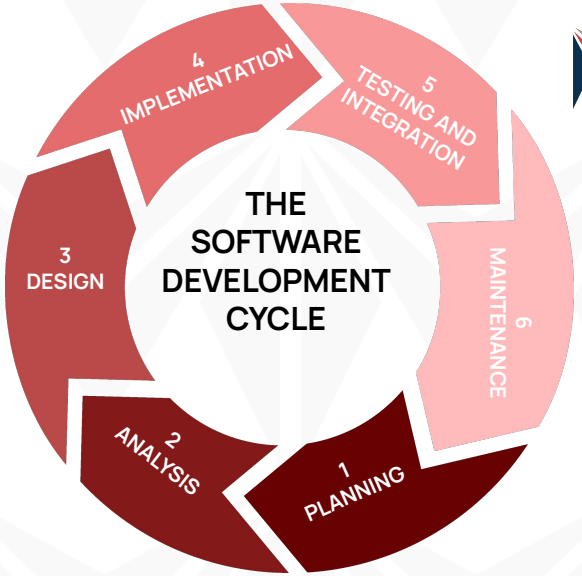


SOFTWARE DEVELOPMENT PROCESS

The software development process defines a structured approach to building software.

Key phases in the software development process

- Planning
- Analysis
- Design
- Implementation
- Testing
- Maintenance



PHASES OF THE SOFTWARE DEVELOPMENT PROCESS

1. Planning Phase involves:

- setting project goals and objectives,
- defining scope, requirements, and constraints,
- creating project plans and schedules.

2. Analysis Phase involves:

- gathering and documenting user requirements,
- analysing existing systems and processes,
- defining use cases and user stories.

PHASES OF THE SOFTWARE DEVELOPMENT PROCESS

3. Design Phase involves:

- designing the system architecture,
- creating high-level and low-level design,
- UI/UX design and wireframing.

4. Implementation Phase involves:

- coding and programming,
- collaborative development using version control systems,
- best practices in coding standards.

PHASES OF THE SOFTWARE DEVELOPMENT PROCESS

5. Testing Phase involves:

- different types of testing such as unit, integration, system, user acceptance,
- identifying and fixing bugs and issues.

6. Maintenance Phase involves:

- ongoing support, updates, and improvements,
- addressing user feedback,
- software documentation and knowledge transfer.

SOFTWARE DEVELOPMENT METHODOLOGIES

Waterfall

Linear approach, suitable for well-defined projects.

Agile

Iterative and collaborative approach, flexible to changes.

Scrum

Framework for Agile development with sprints.

DevOps

Integration of development and operations for continuous delivery.

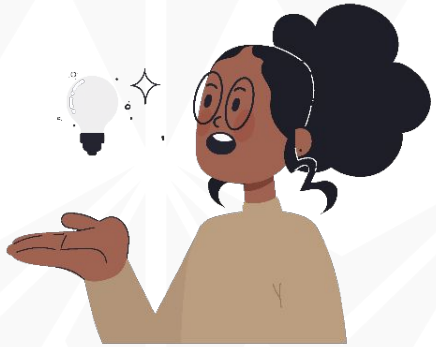


MID-LESSON QUESTION

1. Imagine you're leading a team in developing a complex software application. Discuss how the phases of software development (SDLC) would impact the creation of an innovative social media platform. Address how each phase, from requirements gathering to deployment, contributes to the success of the project. In particular, elaborate on the challenges and strategies you would employ to ensure effective communication, user-centered design, and seamless collaboration across the team during each phase.

ANSWER TO MID-LESSON QUESTION

1. Requirements Gathering: During this phase, the team would gather detailed requirements from stakeholders
2. System Design: In this phase, the architecture of the social media platform is designed.



ANSWER TO MID-LESSON QUESTION

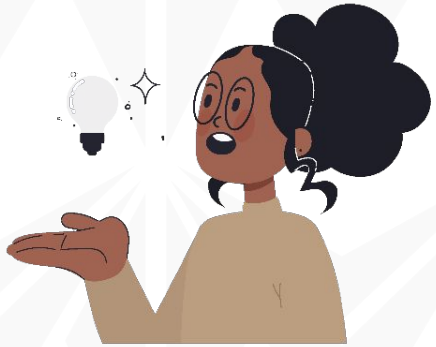
3. Implementation: Developers begin coding based on the design specifications.
4. Testing: Thorough testing is critical to identify bugs and ensure the platform's functionality aligns with the requirements.
5. Deployment: Deploying the social media platform to production requires coordination between development and operations teams.



ANSWER TO MID-LESSON QUESTION

6. Maintenance and Iteration: Once the platform is live, regular maintenance and updates are necessary.

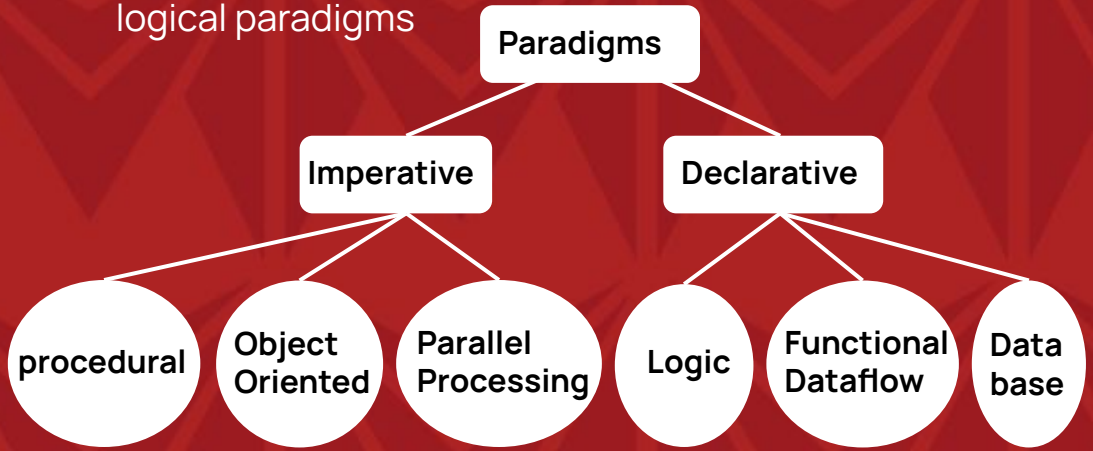
Challenges: Miscommunication, neglecting user experience and disconnected teams
Strategies: Hold regular status meetings, involve UX designers from the beginning and implement collaboration tools for project management and version control.



PROGRAMMING PARADIGMS

Programming Paradigms such as Imperative, Functional, Declarative and Logical programming describes the different approaches to writing code.

Our focus will be on functional, declarative, and logical paradigms



DECLARATIVE PROGRAMMING

A programming paradigm that focuses on expressing the logic and rules of a computation without explicitly detailing the control flow.

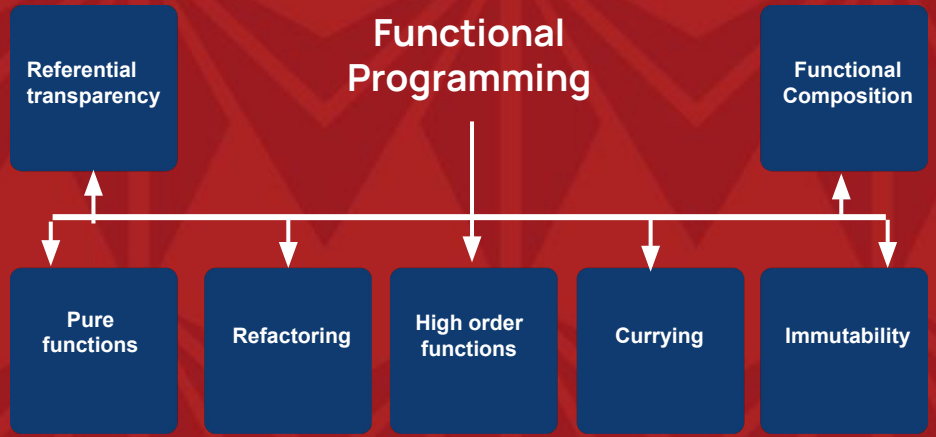
Instead of providing step-by-step instructions for how to achieve a result, you define what you want to achieve, and the underlying system figures out how to accomplish it.

This approach is often used in languages like SQL, HTML, and some aspects of functional and logical programming.



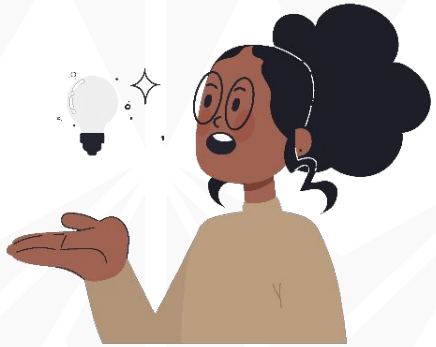
FUNCTIONAL PROGRAMMING

Functional programming is a programming paradigm that treats computation as the evaluation of mathematical functions and avoids changing state and mutable data.



FUNCTIONAL PROGRAMMING

It is based on the concept that functions can be passed as arguments to other functions, returned as values from functions, and assigned to variables.



FUNCTIONAL PROGRAMMING

Functional Programming has core principles such as immutability, pure functions and higher-order functions.

Examples of functional programming languages: Haskell, Scala, Clojure.



BENEFITS OF FUNCTIONAL PROGRAMMING

- Improved code quality and maintainability.
- Readable and concise code.
- Better support for parallel processing



LOGICAL PROGRAMMING

Logical programming is based on formal logic and revolves around the concept of defining relationships, rules, and facts.

In logical programming languages like Prolog, relationships are expressed between different entities as well as the conditions under which certain conclusions can be drawn.

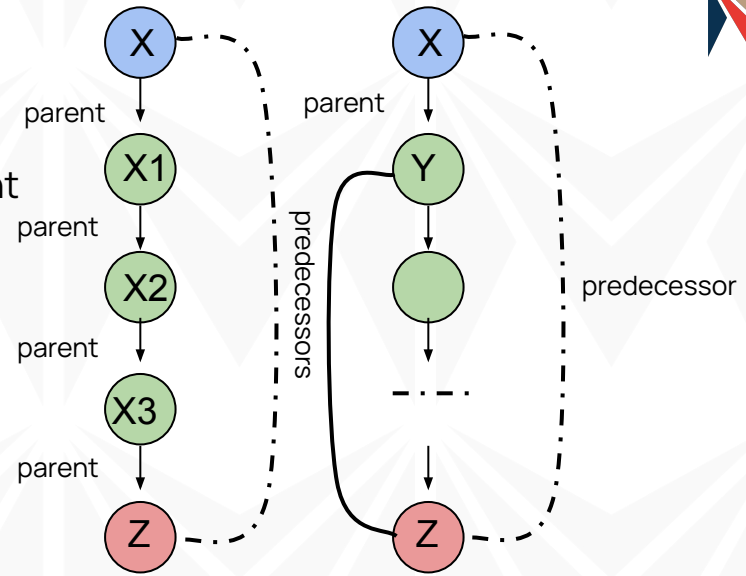
Prolog is a well-known logical programming language that allows you to define facts, rules, and queries



BASIC EXAMPLE IN PROLOG

In the example below, we define facts about parent relationships and rules to determine ancestral relationships.

```
parent(john, alice).
parent(john, bob).
parent(alice, carol).
ancestor(X, Y) :- parent(X, Y).
ancestor(X, Y) :- parent(X, Z), ancestor(Z, Y).
```



IMPERATIVE PROGRAMMING

Imperative programming is a programming paradigm that focuses on describing the sequence of steps that a program must take to reach a certain state or achieve a specific goal.

In imperative programming, you provide explicit instructions to the computer on how to perform a task, step by step.

This paradigm is closely related to the way most people think about solving problems: breaking them down into smaller, manageable steps.



IMPERATIVE PROGRAMMING EXAMPLE

```
public class IP Example {
   public static int factorial(int n) {
      int result = 1;
      for (int i = 1; i < = n; i++) {
         result *= i;
      }
      return result;
   }
   public static void main(String[] args) {
      int num = 5;
      int factorialOfNum = factorial(num);
      System.out.println("The factorial of " + num + " is " +
factorialOfNum);
   }
}
```

REAL-WORLD USE CASES

Declarative programming in database management and query optimization.

Functional programming in finance, data analysis, and scientific computing.

Logic programming for artificial intelligence and knowledge representation.



CHOOSING THE RIGHT PARADIGM

Choosing a programming paradigm is an important decision that can significantly influence how we approach and solve problems in software development.

Different programming paradigms offer different ways of structuring code and expressing solutions. We will see some key considerations to keep in mind when choosing a programming paradigm:

CHOOSING THE RIGHT PARADIGM

Problem Domain and Requirements

Some paradigms are better suited for certain types of problems. For example, object-oriented programming (OOP) is often a good fit for modeling real-world entities and relationships, while functional programming (FP) can be beneficial for tasks involving data transformations and calculations.



CHOOSING THE RIGHT PARADIGM

Team Experience and Expertise

If the team is experienced in a particular paradigm, it might be more efficient to continue using it. On the other hand, if you're introducing a new paradigm, you'll need to consider the learning curve and potential training required.



CHOOSING THE RIGHT PARADIGM

Scalability and Maintainability

Consider how well the chosen paradigm supports scalability and maintainability. Some paradigms, like OOP, emphasize modularity and encapsulation, which can make it easier to manage larger codebases over time.



CHOOSING THE RIGHT PARADIGM

Performance and Efficiency

Different paradigms can have varying impacts on the performance of software. For example, low-level languages and imperative programming might offer better control over memory usage and performance optimization compared to more abstract paradigms like functional programming.



CHOOSING THE RIGHT PARADIGM

Readability and Expressiveness

Some paradigms promotes code that is more readable and expressive, making it easier to understand and maintain. For instance, declarative programming paradigms focus on describing the "what" of a computation rather than the "how," which can lead to more concise and comprehensible code.



CHOOSING THE RIGHT PARADIGM

Compatibility and Integration

Consider the existing tools, libraries, and frameworks that you plan to use. Certain paradigms might be more aligned with the ecosystem you are working in, making integration smoother.



Summary

SUMMARY

- A well-defined software development process is crucial for successful projects.
- Different methodologies cater to diverse project requirements.
- Each phase plays a vital role in delivering reliable software.
- Different programming paradigms offer diverse approaches to solving problems.
- Understanding and mastering these paradigms enhances programming skills.
- Choose the right paradigm based on project needs and problem characteristics.

FURTHER READING RESOURCES

- Suoheimo, M. (2023). Service Design for Emerging Technologies Product Development: Bridging the Interdisciplinary Knowledge Gap. Switzerland: Springer International Publishing.
- Soft Computing Methods for System Dependability. (2019). United States: IGI Global.
- Roebuck, K. (2011). Systems Development Life Cycle (SDLC): High-impact Strategies - What You Need to Know: Definitions, Adoptions, Impact, Benefits, Maturity, Vendors. (n.p.): Emereo Pty Limited.

FURTHER READING RESOURCES

- Davis, J., Daniels, R. (2016). Effective DevOps: Building a Culture of Collaboration, Affinity, and Tooling at Scale. United States: O'Reilly Media.
- Principles and Practice of Declarative Programming. (1999). Germany: Springer.
- Braßel, B. (2012). Implementing Functional Logic Programs: By Translation Into Purely Functional Programs. Germany: KS Omniscriptum Publishing.
- Kaisler, S. H. (2005). Software Paradigms. Germany: Wiley.

THANK YOU